Documentation Technique

Sprint 1 — Authentification & Sécurité

Application de **Gestion de Stages**

Stack : Spring Boot 3 · React 18 · PostgreSQL

Établissement	ENSAM — Meknès
Filière	ILSI — 2ème année Cycle Ingénieur
Module	Frameworks J2EE et Spring
Encadrant	Professeur S. Amri
Année univ.	2025 / 2026

Ce document présente, sans code source, l'architecture et le rôle de chaque composant développé lors du Sprint 1 du projet full-stack.

1 Back-end — Vue d'ensemble

1.1 Organisation en couches

Le back-end Spring Boot est organisé en couches responsables chacune d'un rôle précis. Cette séparation garantit que chaque partie du code peut évoluer indépendamment des autres.

Couche	Responsabilité
Contrôleur	Reçoit les requêtes HTTP, valide la forme des données, délègue au service, retourne la réponse HTTP.
DTO	Représente la structure exacte des données envoyées ou reçues par l'API. Isole l'API des entités internes.
Service	Contient toute la logique métier (vérification d'unicité, encodage du mot de passe, attribution des rôles).
Repository	Seule couche autorisée à parler à la base de données. Fournit des méthodes de recherche et d'écriture.
Entité	Représentation Java d'une table de la base de données. Hibernate traduit cette classe en SQL automatiquement.
Sécurité	Filtre chaque requête entrante, applique les règles d'accès, configure le chiffrement et le CORS.

2 Back-end — Couche Contrôleur

2.1 Rôle général du contrôleur

Le contrôleur est la *porte d'entrée* de l'application. C'est lui qui reçoit les requêtes HTTP venant du navigateur, les lit, et décide quoi en faire. Il ne contient aucune logique métier : il délègue immédiatement au service concerné.

2.2 AuthController — Contrôleur d'authentification

`AuthController` est le contrôleur dédié à toutes les opérations d'authentification. Il écoute les requêtes arrivant sur le chemin `/api/auth` et expose deux fonctionnalités.

Route d'inscription — POST /api/auth/signup

Cette route reçoit les informations du futur utilisateur (nom d'utilisateur, adresse e-mail, mot de passe, rôle choisi). Elle vérifie d'abord que toutes les données sont bien formées grâce aux règles de validation définies dans le DTO. Si tout est conforme, elle transmet ces données au service qui se charge de créer le compte en base. En cas de succès, elle répond au client avec un message de confirmation.

Route de connexion — POST /api/auth/signin

Cette route reçoit un nom d'utilisateur et un mot de passe. Elle est destinée à vérifier l'identité de l'utilisateur et à lui délivrer un *jeton JWT* (JSON Web Token) lui permettant d'accéder aux parties protégées de l'application. La logique

complète de génération du jeton sera finalisée au Sprint 3.

Pourquoi séparer inscription et connexion ?

Les deux opérations ont des besoins très différents. L'inscription crée un compte (opération d'écriture) et accepte plus de champs (email, rôle). La connexion vérifie une identité (opération de lecture) et ne reçoit que le nom d'utilisateur et le mot de passe.

3 Back-end — Couche DTO

3.1 Qu'est-ce qu'un DTO ?

Un *Data Transfer Object* (DTO) est un objet dont le seul but est de transporter des données entre le client et le serveur. Il définit exactement ce que l'API accepte en entrée, avec les contraintes de validation associées. Les DTOs protègent les entités JPA d'une exposition directe.

3.2 LoginRequest — DTO de connexion

`LoginRequest` représente le corps de la requête de connexion. Il contient uniquement deux informations : le nom d'utilisateur et le mot de passe. Chacun de ces champs doit être renseigné — une valeur vide ou absente est immédiatement rejetée par Spring avant même d'atteindre le contrôleur.

Champ	Type	Contrainte
`username`	Texte	Obligatoire, non vide
`password`	Texte	Obligatoire, non vide

3.3 SignupRequest — DTO d'inscription

`SignupRequest` représente le corps de la requête d'inscription. Il contient plus d'informations que le DTO de connexion, et porte des contraintes plus précises sur chaque champ.

Champ	Type	Contrainte
`username`	Texte	Obligatoire · entre 3 et 20 caractères
`email`	Texte	Obligatoire · format e-mail valide · max 50 caractères
`password`	Texte	Obligatoire · entre 6 et 40 caractères
`roles`	Liste	Optionnel · ex : `["ROLE_ETUDIANT"]`

Validation automatique

Toutes ces contraintes sont vérifiées automatiquement par Spring Boot avant que le contrôleur ne soit exécuté. Si une contrainte est violée (champ vide, email mal

formé, mot de passe trop court), Spring retourne directement une erreur HTTP 400 au client, sans toucher à la base de données.

4 Back-end — Couche Entité (JPA)

4.1 Rôle des entités

Les entités JPA sont des classes Java qui représentent des *tables de base de données*. Grâce à Hibernate (le moteur JPA utilisé par Spring Boot), ces classes sont automatiquement traduites en tables SQL lors du démarrage de l'application. Modifier une classe entité modifie la structure de la table correspondante.

4.2 ERole — Énumération des rôles

`ERole` est une énumération Java qui liste les rôles possibles dans l'application. Elle garantit que seules des valeurs prédéfinies peuvent être utilisées comme rôle — ce qui évite les erreurs de saisie et facilite les contrôles d'accès.

Valeur	Signification
`ROLE_ETUDIANT`	Étudiant cherchant un stage
`ROLE_RECRUTEUR`	Entreprise proposant des offres de stage
`ROLE_ADMIN`	Administrateur de la plateforme

4.3 Role — Entité de rôle

`Role` est la classe Java qui correspond à la table `roles` en base de données. Chaque ligne de cette table représente un rôle disponible dans l'application. La table est peuplée au démarrage avec les trois rôles définis dans **`ERole`**. Le nom du rôle est stocké sous forme de texte (et non sous forme de numéro) pour garantir la lisibilité et la stabilité des données.

Colonne	Type SQL	Rôle
`id`	Entier auto-incrémenté	Clé primaire
`name`	Texte (max 20 car.)	Nom du rôle (`ROLE_ETUDIANT`, etc.)

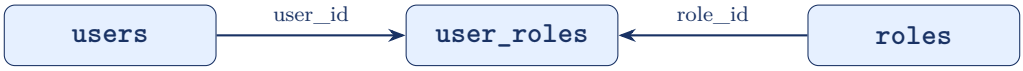
4.4 User — Entité utilisateur

`User` est la classe centrale de l'application. Elle correspond à la table `users` et stocke les informations de chaque compte. Cette entité est liée à **`Role`** par une relation *ManyToMany* : un utilisateur peut avoir plusieurs rôles, et un même rôle peut être partagé par plusieurs utilisateurs.

Colonne	Type SQL	Rôle
`id`	Entier auto-incrémenté	Clé primaire
`username`	Texte	Identifiant unique de l'utilisateur
`email`	Texte	Adresse e-mail
`password`	Texte	Mot de passe **chiffré** (BCrypt)

4.4.1 La relation ManyToMany et la table de jointure

La relation entre **`User`** et **`Role`** ne peut pas être représentée directement dans les deux tables. JPA crée automatiquement une *table de jointure* appelée `user_roles` qui stocke les paires (utilisateur, rôle). Cette table contient deux colonnes : `user_id` et `role_id`, qui pointent respectivement vers les tables `users` et `roles`.



Sécurité du mot de passe

Le champ `password` ne stocke **jamais** le mot de passe en clair. Avant la persistance, le service appelle l'encodeur BCrypt qui transforme le mot de passe en une empreinte cryptographique irréversible. Même si la base de données est compromise, les mots de passe restent illisibles.

5 Back-end — Couche Repository

5.1 Rôle général des repositories

Les repositories sont les composants responsables de toute communication avec la base de données. Ils constituent une *couche d'abstraction* qui évite d'écrire du SQL manuellement. Spring Data JPA génère automatiquement les requêtes SQL à partir des noms des méthodes déclarées dans l'interface.

5.2 RoleRepository

`RoleRepository` donne accès aux données de la table `roles`. Il est principalement utilisé par le service lors de l'inscription : quand un utilisateur choisit le rôle `ROLE_ETUDIANT`, le service interroge ce repository pour récupérer l'objet **`Role`** correspondant depuis la base, puis l'associe à l'utilisateur.

La méthode `findByName` effectue une recherche par nom de rôle. Si aucun rôle correspondant n'est trouvé, elle retourne un résultat vide (de type `Optional`), ce qui permet de gérer l'erreur proprement.

5.3 UserRepository

`UserRepository` donne accès aux données de la table `users`. Il expose trois fonctionnalités essentielles pour la gestion des comptes :

Méthode	Utilisation
`findByUsername`	Retrouve un utilisateur par son nom d'utilisateur. Utilisé lors de la connexion pour charger le profil.
`existsByUsername`	Vérifie si un nom d'utilisateur est déjà pris avant l'inscription. Renvoie `true` ou `false`.
`existsByEmail`	Vérifie si une adresse e-mail est déjà utilisée avant l'inscription. Empêche les doublons.

Convention Spring Data JPA

Spring Data déduit la requête SQL depuis le nom de la méthode. `findByUsername` génère `SELECT * FROM users WHERE username = ?`, et `existsByEmail` génère `SELECT COUNT(*) > 0 FROM users WHERE email = ?`. Aucune ligne de SQL n'est écrite manuellement par le développeur.

6 Back-end — Sécurité (Spring Security)

6.1 Rôle de WebSecurityConfig

`WebSecurityConfig` est le fichier de configuration central de Spring Security. Il définit les règles de sécurité appliquées à **toutes** les requêtes entrantes : qui peut accéder à quoi, comment les sessions sont gérées, quelles origines sont autorisées à appeler l'API.

6.2 Gestion des mots de passe — BCrypt

La configuration déclare un *encodeur BCrypt* comme composant Spring. BCrypt est un algorithme de hachage spécialement conçu pour les mots de passe : il est intentionnellement lent (pour résister aux attaques par force brute) et intègre un mécanisme de sel aléatoire (pour que deux mots de passe identiques produisent des empreintes différentes). Cet encodeur est injecté dans le service d'inscription pour chiffrer le mot de passe avant la sauvegarde.

6.3 Règles d'accès aux endpoints

La configuration définit deux niveaux d'accès :

Chemin	Accès	Raison
`/api/auth/**`	Public (sans token)	Inscription et connexion doivent être accessibles sans être
Tout autre chemin	Authentifié requis	Protège les données métier

6.4 Sessions stateless

L'application est configurée pour ne créer **aucune session HTTP** côté serveur. Chaque requête est autonome et s'identifie avec son propre jeton JWT dans l'en-tête. Ce choix est cohérent avec une API REST consommée par une Single Page Application React.

6.5 Configuration CORS

CORS (*Cross-Origin Resource Sharing*) est un mécanisme de sécurité des navigateurs qui bloque par défaut les appels HTTP vers un domaine différent de celui de la page web. Puisque React tourne sur `localhost:5173` (ou `:5174`) et Spring Boot sur `localhost:8080`, les deux domaines sont techniquement différents.

La configuration CORS déclare explicitement :

— Les **origines autorisées** : `localhost:5173` et `localhost:5174`

— Les **méthodes HTTP** autorisées : GET, POST, PUT, DELETE, OPTIONS
— Les **en-têtes** autorisés : `Authorization` (pour le JWT) et `Content-Type` (pour le JSON)

6.6 Désactivation de la protection CSRF

La protection CSRF (*Cross-Site Request Forgery*) est conçue pour les applications web traditionnelles qui utilisent des cookies de session. Notre application utilisant JWT dans les en-têtes HTTP, cette protection est inutile et désactivée pour ne pas perturber les appels API.

7 Back-end — Configuration de l'application

7.1 Rôle du fichier application.properties

Ce fichier centralise toute la configuration de l'application Spring Boot : connexion à la base de données, comportement de JPA, dialecte SQL. C'est le seul endroit à modifier pour passer d'un environnement de développement à un environnement de production.

7.2 Connexion à PostgreSQL

La configuration spécifie l'URL JDBC de la base de données (`gestion_stages_db` sur le port `5432`), le nom d'utilisateur et le mot de passe PostgreSQL. Le pilote JDBC PostgreSQL est déclaré pour que Spring sache comment dialoguer avec ce moteur de base de données.

7.3 Comportement de JPA/Hibernate

Paramètre	Signification
`ddl-auto=update`	Hibernate crée les tables si elles n'existent pas encore, et les met à jour si le modèle Java évolue. En production, on préférera des outils de migration dédiés (Flyway).
`show-sql=true`	Toutes les requêtes SQL générées par Hibernate sont affichées dans les logs — utile pour déboguer en développement.
`format_sql=true`	Les requêtes SQL dans les logs sont indentées et lisibles.
Dialecte PostgreSQL	Indique à Hibernate la syntaxe SQL spécifique à PostgreSQL (types de données, séquences, etc.).

Sécurité en production

Les mots de passe de base de données ne doivent jamais être écrits en clair dans ce fichier si le dépôt est public. En production, il faut utiliser des variables d'environnement ou un service de gestion de secrets.

8 Front-end — Vue d'ensemble

8.1 Organisation du projet React

Le front-end est une *Single Page Application* (SPA) construite avec React et Vite. L'organisation des fichiers suit une logique de séparation des responsabilités.

Dossier / Fichier	Rôle
`api/axios.js`	Configuration Axios partagée + intercepteur JWT
`services/auth.service.js`	Fonctions d'appels API pour l'authentification
`pages/Login.jsx`	Page de connexion (formulaire)
`pages/Register.jsx`	Page d'inscription (formulaire + sélection de rôle)
`App.jsx`	Composant racine et routeur principal

9 Front-end — Configuration des appels HTTP

9.1 api/axios.js — Instance Axios partagée

Ce fichier crée une *instance Axios* configurée une seule fois et réutilisée partout dans l'application. Plutôt que d'utiliser Axios directement dans chaque composant, tous les appels HTTP passent par cette instance commune.

9.1.1 L'intercepteur de requête

L'intercepteur est la fonctionnalité la plus importante de ce fichier. C'est un mécanisme qui s'exécute **automatiquement avant chaque appel HTTP** émis par l'application. Son rôle est de lire le jeton JWT stocké dans le navigateur (s'il existe) et de l'ajouter à l'en-tête `Authorization` de la requête.

Cela évite d'avoir à écrire manuellement ce mécanisme dans chaque composant qui appelle l'API. Le serveur Spring Boot recevra donc automatiquement le jeton d'identification de l'utilisateur connecté sur toutes les requêtes.

Pourquoi stocker le token dans le navigateur ?

Après une connexion réussie, le serveur retourne un jeton JWT. Ce jeton est conservé dans le `localStorage` du navigateur pour persister entre les rechargements de page. L'intercepteur Axios le récupère à chaque requête pour prouver au serveur que l'utilisateur est bien authentifié.

10 Front-end — Service d'authentification

10.1 services/auth.service.js — Centralisation des appels API

Ce fichier regroupe toutes les fonctions de communication avec les endpoints d'authentification du back-end. Les composants React n'appellent jamais Axios directement — ils passent systématiquement par ce service. Cela centralise la logique réseau et facilite la maintenance.

Fonction	Rôle
`register()`	Envoie les données d'inscription au serveur (`POST /api/auth/signup`).
`login()`	Envoie les identifiants au serveur (`POST /api/auth/signin`). Si la connexion réussit, stocke l'objet utilisateur (incluant le jeton JWT) dans le `localStorage`.
`logout()`	Supprime l'objet utilisateur du `localStorage`, déconnectant effectivement l'utilisateur côté client.
`getCurrentUser()`	Lit et retourne l'objet utilisateur stocké dans le `localStorage`. Retourne `null` si personne n'est connecté.

11 Front-end — Pages React

11.1 pages/Login.jsx — Page de connexion

Ce composant React affiche le formulaire de connexion et gère son comportement. Il utilise le *hook useState* pour maintenir en mémoire les valeurs saisies dans les champs en temps réel (chaque frappe au clavier met à jour l'état local du composant).

11.1.1 Comportement à la soumission

Quand l'utilisateur clique sur « Se connecter », le composant appelle la fonction `login()` du service d'authentification. Deux cas se présentent :

- **Succès :** L'utilisateur est redirigé vers la page d'accueil grâce au *hook useNavigate* de React Router.
- **Échec :** Un message d'erreur s'affiche sous le formulaire. Aucune redirection n'a lieu.

11.2 pages/Register.jsx — Page d'inscription

Ce composant affiche le formulaire d'inscription. Il regroupe tous les champs dans un unique objet d'état (`formData`) pour simplifier la gestion du formulaire. Un menu déroulant permet à l'utilisateur de choisir son rôle (`ROLE_ETUDIANT` ou `ROLE_RECRUTEUR`) lors de la création de son compte.

11.2.1 Comportement à la soumission

À la soumission, le composant appelle la fonction `register()` du service. Le rôle sélectionné est transmis sous forme de liste (comme attendu par le back-end). Deux cas se présentent :

- **Succès :** Un message de confirmation vert s'affiche invitant l'utilisateur à se connecter.
- **Échec :** Le message d'erreur retourné par le serveur (par exemple : « nom d'utilisateur déjà pris ») s'affiche en rouge.

Rôle ROLE_ADMIN non proposé

Le rôle administrateur n'est pas proposé dans le formulaire d'inscription public. Il ne peut être attribué que manuellement en base de données ou via un endpoint réservé, afin d'éviter qu'un utilisateur malveillant s'attribue des droits élevés.

12 Front-end — Routeur principal

12.1 App.jsx — Point d'entrée de l'application

`App.jsx` est le composant racine de l'application React. Il joue deux rôles : encapsuler toute l'application dans un *Router* et définir la table de correspondance entre les URLs et les composants à afficher.

12.1.1 Routes définies

URL	Composant affiché	Accès
`/`	Page d'accueil (message de bienvenue)	Public
`/login`	**`Login.jsx`**	Public
`/register`	**`Register.jsx`**	Public

Toutes les routes sont actuellement publiques. Les *routes privées* (qui vérifient si l'utilisateur est connecté avant d'afficher la page) seront mises en place au Sprint 3, en parallèle de l'implémentation complète du JWT.

13 Synthèse et perspectives

13.1 Récapitulatif des composants produits

Composant	Couche	Rôle résumé
`AuthController`	Contrôleur	Expose les routes /signup et /signin
`LoginRequest`	DTO	Valide les données de connexion
`SignupRequest`	DTO	Valide les données d'inscription
`ERole`	Entité	Définit les rôles disponibles
`Role`	Entité JPA	Table `roles` en base
`User`	Entité JPA	Table `users` + relation ManyToMany
`RoleRepository`	Repository	Accès aux rôles en base
`UserRepository`	Repository	Accès aux utilisateurs en base
`WebSecurityConfig`	Sécurité	CORS, accès, sessions, BCrypt
`application.properties`	Config	Connexion PostgreSQL + JPA
`axios.js`	API	Instance Axios + intercepteur JWT
`auth.service.js`	Service	Fonctions login / register / logout
`Login.jsx`	Page	Formulaire de connexion
`Register.jsx`	Page	Formulaire d'inscription
`App.jsx`	Routeur	Routes publiques